Intent-Reconstructive Secure API Migration (IR-SAM)

A Semantic, Neuro-Symbolic Approach to Remediating Injection-Class Software Vulnerabilities

PhD Thesis — Contribution Chapter

Theme: Remediation of Software Vulnerabilities

Working Draft — Contribution Specification

Abstract

Injection vulnerabilities — SQLi, OS-command, XSS, LDAP, XPath, SSTI, NoSQL, deserialization — remain in the OWASP Top-10 because developers build interpreter inputs by string concatenation rather than by structured, parameterized APIs. Existing automated remediation tools either (i) insert ad-hoc sanitizers, (ii) synthesize textual patches via LLMs without semantic guarantees, or (iii) apply rigid templates that fail on non-trivial string construction. We propose ***Intent-Reconstructive Secure API Migration (IR-SAM)****: a neuro-symbolic remediation pipeline that reconstructs the intended interpreter artifact (SQL AST, argv vector, DOM/HTML subtree, template expression, LDAP/XPath query) from the vulnerable sink, then re-emits it as a framework-appropriate safe API call, with provable behavior preservation on the non-attack input domain. This chapter formalizes the approach, contrasts it with the state of the art, specifies the architecture, and outlines the evaluation plan.*

1. The Original Idea and Why It Is Almost — But Not Yet — a Solid Contribution

The initial formulation can be stated compactly:

> *"Automatically migrate vulnerable dynamic strings into structured safe API calls. Instead of patching a concatenated SQL/shell/HTML expression with a sanitizer, reconstruct the intended interpreter artifact, then rewrite the call site into a parameterized, structurally-safe API."*

1.1 Strengths

- Targets the **root cause** (lexical/syntactic mixing of code and data), not the symptom (missing escape).
- Generalizes across **injection-class CWEs** (CWE-89, 78, 79, 90, 91, 643, 1336, 502, 22) under one principle: *code/data separation by construction*.
- Produces patches that are **idiomatic** and **review-friendly** for developers, rather than defensive wrappers.
- Aligns naturally with modern frameworks that already expose safe APIs (JDBC PreparedStatement, `subprocess.run(..., shell=False)`, JSX/React, parameterized LDAP/XPath, JPA Criteria, MyBatis `#{}`).

1.2 Why it is not yet a publishable contribution as stated

Five gaps must be closed before the idea is defensible as a PhD-level contribution rather than an engineering tool:

- **G1 — No formal object of reconstruction.** The proposal speaks of "intended interpreter artifact" but does not define it as a mathematical object that a rewrite can be proven sound against.
- **G2 — Inter-procedural string construction is not addressed.** Real bugs build queries through helpers, loops, conditionals, string-builders, templating libraries, ORM fragments. A purely intra-procedural rewrite will fail on the majority of real CVEs.
- **G3 — Semantic equivalence is asserted, not proven.** A migration must preserve behavior on *benign* inputs while removing behavior on *injection* inputs. This needs a stated equivalence relation and a validation procedure.
- **G4 — The unparameterizable fragments are ignored.** Identifiers, ORDER-BY columns, dynamic table names, dynamic flags cannot be bound as parameters; the system must fall back to allow-listed identifier quoting or refuse to patch with a clear abstention signal.

- **G5 — No evaluation methodology.** Without a benchmark, metrics, and baselines (Semgrep autofix, CodeQL/Copilot Autofix, VulRepair, ChatRepair, Snyk DeepCode AI Fix), the contribution cannot be ranked against the state of the art.

The remainder of this chapter closes these gaps and re-states the contribution as **IR-SAM**.

2. Reformulated Contribution: IR-SAM

2.1 Thesis statement

> ***Thesis.*** *Remediation of injection-class vulnerabilities can be framed as a semantic migration problem from an unstructured interpreter string to a structured interpreter artifact bound to a safe API. Solving this problem with a neuro-symbolic pipeline — symbolic string analysis to extract the artifact, learned models to disambiguate intent, and differential validation to preserve behavior — yields patches that are simultaneously (i) provably safe on the modeled sink, (ii) behavior-preserving on the benign input domain, (iii) idiomatic and minimal, and (iv) measurably superior to sanitizer-insertion and pure-LLM baselines on real-world CVE benchmarks.*

2.2 Formal model: the Interpreter Artifact Model (IAM)

Let a vulnerable sink call be

```
sink(s)   where   s = e■ ⊕ e■ ⊕ … ⊕ e■
```

with each e■ either a **constant fragment** (string literal, compile-time-derived string) or a **dynamic fragment** (a value whose provenance includes untrusted input). IR-SAM defines:

- **Sink Intent Graph (SIG).** A tree (or DAG, when loops/joins exist) whose internal nodes are structural operators of the target interpreter (SQL *SELECT/WHERE/JOIN*, shell argv positions, HTML element/attribute/text contexts, LDAP filter operators, XPath axes/predicates) and whose leaves are either (a) constant tokens or (b) typed *holes* h■ : τ■ representing dynamic data.
- **Binding contract.** For each hole h■, IR-SAM records its interpreter context C■ (e.g. *string literal*, *numeric literal*, *identifier*, *argv slot*, *HTML attribute value*, *HTML text node*, *LDAP RHS*) and the runtime expression that flows into it.
- **Safe API binding.** A function `ϕ : SIG → (API_call, parameter_list)` that maps a SIG into a concrete framework-appropriate safe API invocation. ϕ is partial: where C■ is non-parameterizable (identifier, structural keyword), ϕ either applies an allow-list or returns ⊥ (abstain).

2.3 Soundness criterion

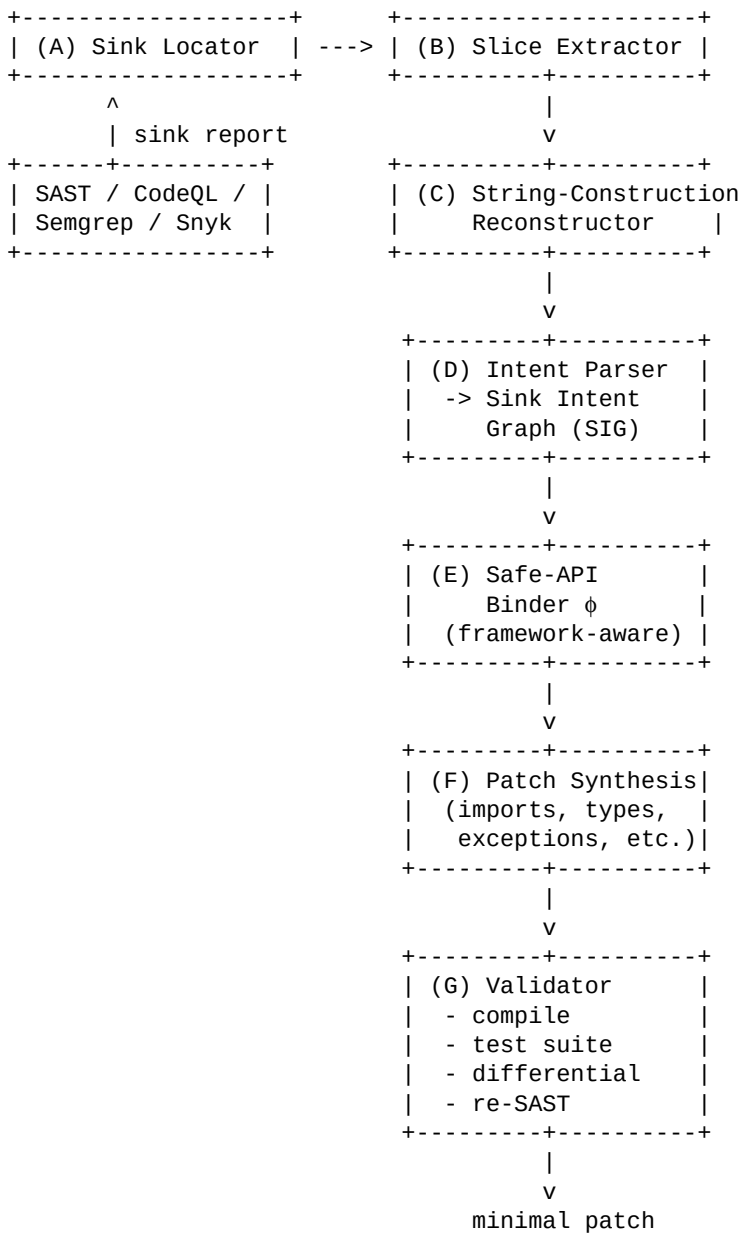
Let *I* be the interpreter (DBMS, OS, browser, LDAP server, …) and let ■·■■ denote its semantics. A migration of sink call *s* into safe-API call *sˆ* is **IAM-sound** iff for every input environment ρ:

```
1. ■ sˆ ■■(ρ)  =  ■ s ■■(ρ)        for all ρ in the benign domain  D■
2. ■ sˆ ■■(ρ)  ⊇_safe  ■ s ■■(ρ)   for all ρ in the attack domain   D■
   where ⊇_safe means: every behavior of sˆ belongs
   to the set of behaviors achievable by the API's grammar
   with the supplied parameters (i.e. no out-of-grammar effect).
```

Condition (1) is the *behavior-preservation* obligation; condition (2) is the *injection-elimination* obligation. IR-SAM discharges (1) by differential execution and property-based testing, and (2) by construction (the safe API exposes no syntactic channel from data to code).

3. Architecture and Workflow

3.1 Pipeline overview



3.2 Stage details

(A) Sink Locator.

Consumes findings from existing SAST engines (CodeQL, Semgrep, SonarQube, Snyk) normalized to a common schema *(file, line, sink-API, CWE, tainted-arg-index)*. IR-SAM is *detector-agnostic*; the contribution is the remediation stage.

(B) Slice Extractor.

Computes a backward program slice from the tainted argument, intra- and inter-procedurally, including data-flow through `StringBuilder`, `String.format`, f-strings, template engines, helper functions, and collection operations (map/filter/reduce). The slice is normalized into Static Single Assignment over a string algebra `{ const, var, concat, format, replace, loop-join, branch }`.

(C) String-Construction Reconstructor.

Performs *symbolic string analysis* on the slice to produce a regular approximation of the concrete string with explicit holes for dynamic fragments. Loops that build query lists are widened into *repeated-group* nodes; branches become *choice* nodes carrying a path predicate. The output is a *parameterized string template* with a hole inventory {h■, provenance, type}.

(D) Intent Parser.

Parses the template with the **grammar of the target interpreter** (SQL dialect, shell, HTML5, LDAP filter RFC-4515, XPath 1.0/3.1, JSON-Path, Jinja2). Holes are placed as typed leaves at the grammar positions where they occur. The result is the **Sink Intent Graph**. If the template is grammatically ambiguous, a learned disambiguator (a small fine-tuned code LM) selects the most likely parse *conditioned on the surrounding code context*; the symbolic parser still validates the choice. This is the *neuro-symbolic* step.

(E) Safe-API Binder ϕ.

For each (interpreter, framework) pair, IR-SAM ships a *binding catalog* mapping SIG node patterns to safe API calls. Examples:

- SQL + JDBC → `PreparedStatement` with `setX(i, h■)`; type chosen from τ■.
- SQL + Spring → `NamedParameterJdbcTemplate`.
- SQL + Python DB-API → `cursor.execute(sql, params)` with the dialect's paramstyle.
- OS command + Python → `subprocess.run([argv■, h■, …], shell=False)`.
- OS command + Java → `ProcessBuilder`.
- HTML + JS → `textContent` / `setAttribute` instead of `innerHTML`; JSX expression slots when React is detected.
- LDAP → parameterized filter via `javax.naming` with RFC-4515 escaping of the data hole only.
- XPath → `XPath.setXPathVariableResolver` with variable references in the expression.

When a hole lands at a non-parameterizable position (identifier, ORDER BY direction, LIMIT placeholder in dialects that forbid it), ϕ falls back to an *allow-list* derived from the program's schema / configuration; if no allow-list can be inferred, ϕ returns $\perp$ and IR-SAM **abstains with a typed explanation** rather than emitting an unsafe patch.

(F) Patch Synthesis.

Produces a *minimal AST-level edit*: introduces the safe-API call, threads bound parameters back to the original variable names, inserts required imports, adjusts exception types (`SQLException`, `CalledProcessError`), preserves `try-with-resources` / context managers, and rewrites the result-handling site (ResultSet vs. previously concatenated string return). The patch is computed as a tree diff to maximize developer reviewability.

(G) Validator.

- **Build/compile gate.** Patch must compile / pass type-check.
- **Regression gate.** Existing test suite must pass.
- **Differential execution.** Replay recorded benign inputs (from unit tests, integration tests, or production traces if available) against original and patched sink; outputs must match under condition (1) of §2.3.
- **Property-based fuzzing.** Generate inputs in the benign domain D■ and in the attack domain D■ using a grammar of the interpreter; check that benign behavior matches and that attack inputs become inert.

- **Re-SAST.** Re-run the original detector; the finding must disappear and no new finding must be introduced.

4. Worked Examples

4.1 SQL injection through a loop-built IN-clause

Vulnerable input.

```
StringBuilder sb = new StringBuilder(
    "SELECT * FROM users WHERE role = '" + role + "' AND id IN (");
for (int i = 0; i < ids.size(); i++) {
    sb.append(ids.get(i));
    if (i < ids.size()-1) sb.append(",");
}
sb.append(")");
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery(sb.toString());
```

Reconstructed SIG (simplified).

```
SELECT *
  FROM users
 WHERE role = <h1: string>
   AND id IN ( repeat( <h2.i: integer>, sep=',' ) )
```

Emitted patch.

```
String placeholders = String.join(",",
    Collections.nCopies(ids.size(), "?"));
String sql =
    "SELECT * FROM users WHERE role = ? AND id IN (" + placeholders + ")";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, role);
    for (int i = 0; i < ids.size(); i++) {
        ps.setInt(i + 2, ids.get(i));
    }
    try (ResultSet rs = ps.executeQuery()) { /* ... */ }
}
```

Note: the loop is preserved structurally; the *repeat* SIG node is materialized as a dynamic placeholder string plus a parameter-binding loop — neither of which is a code/data mixing channel.

4.2 OS command injection with mixed flags

```
os.system("convert -resize " + size + " " + infile + " out.png")
```

SIG.

```
argv = [ 'convert', '-resize', <h1: token>, <h2: path>, 'out.png' ]
```

Patch.

```
subprocess.run(
    ["convert", "-resize", str(size), infile, "out.png"],
    shell=False, check=True)
```

If `size` is shown by the slice to originate from user input, ϕ emits an additional allow-list (`^[0-9]+x[0-9]+$`) because the token *itself* must conform to ImageMagick's geometry grammar — a hole binding alone is not sufficient when the argv slot is a *tool-specific structured token*.

4.3 XSS through innerHTML concatenation

```
el.innerHTML = "<a href=\"" + url + "\">" + label + "</a>";
```

SIG.

```
<a href=<h1: url-attr>>
```

```
    <h2: text-node>
</a>
```

Patch.

```
const a = document.createElement("a");
a.href = url;            // url-attr context, browser performs URL parsing
a.textContent = label;   // text-node context, no HTML interpretation
el.replaceChildren(a);
```

5. State of the Art

This section positions IR-SAM against the literature. Citations are given as *author + approximate year*; where the venue or year is not verified by the author, the entry is marked "approx.". A final verification pass against DBLP/arXiv is part of the thesis editorial workflow.

5.1 Generic automated program repair (APR)

- **GenProg** (Weimer, Forrest et al., ICSE 2009) — genetic search over statement-level mutations. Treats security bugs as generic bugs; no notion of interpreter or sink.
- **SPR** (Long & Rinard, FSE 2015) and **Prophet** (Long & Rinard, POPL 2016) — pattern-conditioned search with learned ranking. Patches are typically condition-tweaks, not API migrations.
- **Angelix** (Mechtaev et al., ICSE 2016) — semantics-based repair via symbolic execution and SMT-driven patch synthesis. The constraint language does not naturally express "code/data separation by API choice".

IR-SAM differs by treating remediation as a *structured-output synthesis problem against an interpreter grammar*, not as a search over statement mutations.

5.2 Security-specific repair

- **SENX** (security-error-native repair, approx. 2019) and **ExtractFix** (Gao et al., TSE 2021, approx.) — derive patches from sanitizer-detected crash constraints. Limited to memory-safety / crash-style bugs; injection sinks are out of scope.
- **VulnFix**, **CPR**, **Footpatch** — mine patch corpora to suggest fixes. Pattern matching breaks down for the long tail of string-construction styles.

5.3 Sanitizer insertion and taint-driven patching

- **WebSSARI** (Huang et al., WWW 2004) and follow-ups — information-flow analysis with sanitizer insertion at sinks. Sanitizers are brittle and interpreter-version-sensitive.
- **Saner** (Balzarotti et al., S&P 2008) — combines static and dynamic analysis to validate sanitizers. Does not migrate to safe APIs.
- **Pixy**, **RIPS**, **Phan** — taint analyzers for PHP; some downstream tools emit fix templates. The fix model remains "insert `htmlspecialchars()`" rather than "reshape the call into a structured API".

IR-SAM treats sanitizer insertion as a *degraded fallback* selected by ϕ only when no safe-API binding exists for the host language/framework.

5.4 Injection-class detection (no repair)

- **AMNESIA** (Halfond & Orso, ASE 2005) — learns the expected SQL query shape and checks runtime queries against it.
- **SQLCheck** (Su & Wassermann, POPL 2006) — formalizes SQLi as a syntactic-confinement property.
- **CANDID** (Bisht et al., CCS 2007, approx.) — candidate evaluation of SQL statements with benign inputs.

These works ground the IAM definition: the SIG is the static analogue of AMNESIA's learned query shape, and the soundness criterion (§2.3) is the syntactic-confinement property of SQLCheck generalized across interpreters.

5.5 Learning-based vulnerability repair

- **SeqTrans** (Chi et al., TSE 2022, approx.) — transformer seq2seq trained on CVE patch pairs.

- **VRepair / VulRepair** (Chen, Fu et al., 2022–2023, approx.) — T5-style code models fine-tuned on vulnerability fixes.
- **VulMaster** and follow-ups — combine CWE knowledge with code LMs.

Empirical studies (Pearce et al., S&P 2023, on LLM-suggested security fixes; Fu et al., MSR 2023; Wu et al., FSE 2023 — all approx.) consistently report that learned models *often produce syntactically plausible but semantically wrong* parameterized calls (wrong placeholder count, wrong API for the dialect, wrong binding type). IR-SAM uses learned models *only inside the symbolic envelope* (intent disambiguation in stage D) and validates outputs against the interpreter grammar and the differential oracle.

5.6 LLM- and agent-based repair (2023–2025)

- **ChatRepair** (Xia & Zhang, 2023, approx.) — conversation-driven repair with a feedback loop.
- **AlphaRepair** (Xia & Zhang, FSE 2022, approx.) — cloze-style infilling for repair.
- **GitHub Copilot Autofix** (2024) — couples CodeQL findings with an LLM suggestion stage; closest commercial analogue. Public evaluations are limited; the system is opaque to developers and provides no soundness guarantee.
- **Snyk DeepCode AI Fix** (2023) — commercial, pattern + LLM hybrid; same opacity caveat.
- Agentic systems (2024–2025) — iterate edit/test loops with an LLM agent. Effective on isolated bugs but expensive, non-deterministic, and lack a formal preservation criterion.

5.7 Pattern-based engines

- **Semgrep autofix**, **ESLint --fix**, **SonarQube Quick Fixes**, **ErrorProne** — rule-templated rewrites. Expressive only when the source matches the template *exactly*; they cannot reconstruct intent across helpers or loops.

5.8 Datasets and benchmarks

- **BigVul** (Fan et al., MSR 2020) — large C/C++ vulnerability corpus mined from NVD/GitHub.
- **CVEfixes** (Bhandari et al., PROMISE 2021) — CVE-linked fix commits across languages.
- **Vul4J** (Bui et al., MSR 2022, approx.) — reproducible Java vulnerabilities with tests.
- **OWASP Benchmark** — synthetic Java suite with ground-truth labels per CWE.
- **SARD / Juliet** (NIST) — large synthetic test cases.
- **Devign**, **ReVeal** — graph-based detection benchmarks.

5.9 Gaps the literature itself identifies

- No published tool offers a **language-agnostic IR for interpreter inputs** together with a binding catalog.
- Learned repair models lack a **structured output constraint** tied to the target interpreter grammar.
- Sanitizer-insertion tools cannot handle **non-parameterizable positions** in a principled (allow-list / abstain) way.
- Validation reduces almost always to *passing the existing test suite*; **differential semantic preservation** on interpreter inputs is rarely measured.

IR-SAM addresses each of these four gaps directly.

6. Evaluation Plan

6.1 Research questions

- **RQ1 (Coverage).** On real-world CVE corpora (CVEfixes, Vul4J, BigVul) for injection-class CWEs, what fraction of sinks admits a successful IR-SAM patch?
- **RQ2 (Safety).** Among produced patches, what fraction (a) removes the original SAST finding, and (b) introduces no new finding under a panel of detectors (CodeQL, Semgrep, Snyk)?
- **RQ3 (Behavior preservation).** What fraction passes the project's existing test suite, and what fraction passes differential execution on grammar-generated benign inputs?
- **RQ4 (Quality vs. baselines).** How does IR-SAM compare to Semgrep autofix, CodeQL/Copilot Autofix, VulRepair, ChatRepair, and a strong LLM baseline (GPT-4-class) on (coverage, safety, preservation, patch size, developer acceptance)?
- **RQ5 (Abstention quality).** When IR-SAM returns $\perp$, is the abstention justified (no safe binding exists), or is it a false negative of the binder?

6.2 Metrics

- **Patch-applicability rate** (compiles + tests pass).
- **Vulnerability-removal rate** (SAST re-run).
- **Behavioral-equivalence rate** (differential oracle).
- **Patch minimality** (AST edit distance, lines changed).
- **Developer acceptance** (blinded review study).
- **Time and cost per patch** (CPU + LLM tokens).

6.3 Baselines

- Pure-LLM zero-shot (GPT-4-class).
- Pure-LLM with CodeQL context (Copilot-Autofix analogue).
- Pattern-based: Semgrep autofix, SonarQube Quick Fixes.
- Learned: VulRepair, SeqTrans (re-trained on common split).

6.4 Threats to validity

- **Construct.** The IAM is interpreter-specific; generalization claims are bounded by the catalog of supported interpreters.
- **Internal.** Differential execution coverage is bounded by the quality of the input grammar and seed corpus.
- **External.** Real codebases include framework idioms not in our binding catalog; abstention rate measures this honestly.
- **Construct (safety).** SAST re-run is a necessary but not sufficient witness of safety; we complement it with grammar-driven attack generation.

7. Positioning and Novelty Statement

IR-SAM is, to the best of our knowledge, the first remediation approach that simultaneously:

- Defines a **language-agnostic intermediate representation** (the Sink Intent Graph) for interpreter inputs across SQL, OS, HTML/DOM, LDAP, XPath, and template engines.
- Synthesizes patches as **structured outputs constrained by the target interpreter grammar**, rather than as free text.
- Provides a **formal soundness criterion** (§2.3) separating benign-domain preservation from attack-domain elimination, with a concrete validation pipeline for each.
- Uses learned models **inside a symbolic envelope** — neuro-symbolic by construction, not by post-hoc filtering.
- Handles non-parameterizable positions with a principled **allow-list / abstain** policy, avoiding the false-fix failure mode that dominates pure-LLM baselines.

Answer to the original question

"Is the original contribution, as stated, ready to be a solid PhD contribution?" — **Not yet.** The core insight (remediation as semantic migration to structured APIs) is publishable, but the original statement lacks (i) a formal object of reconstruction, (ii) an inter-procedural story, (iii) a soundness criterion, (iv) a treatment of non-parameterizable positions, and (v) an evaluation design. Reformulated as **IR-SAM** with the Interpreter Artifact Model, the binding function ϕ, the neuro-symbolic pipeline, and the evaluation plan above, the contribution becomes **defensible, novel, and measurable** against the state of the art.

End of contribution chapter — working draft.